UNIVERSITÀ DI BOLOGNA

School of Engineering

Master Degree in Automation Engineering

Optimal Control

Optimal Control Strategies for Quadrotor with Suspended Loads

Professor: **Giuseppe Notarstefano**

Student: **Mayuresh Pachore**

Academic year 2024/2025

Abstract

This project investigates optimal control strategies for a quadrotor equipped with a suspended load, a system characterized by nonlinear and highly coupled dynamics. The study focuses on three methods: Newton's method for trajectory generation, Linear Quadratic Regulator (LQR) for trajectory tracking, and Model Predictive Control (MPC) for predictive constrained control.

Newton's method is applied to compute optimal state-input trajectories between equilibrium points, exploiting second-order information for fast convergence. Both step-based and smooth reference trajectories are evaluated, with results showing that smooth references improve convergence speed and accuracy. Based on these trajectories, the system is linearized and an LQR controller is designed to provide robust tracking and disturbance rejection. Finally, an MPC framework is developed to handle system and input constraints while ensuring smooth control actions. Simulations demonstrate that MPC achieves superior tracking in the presence of disturbances, although with higher computational cost.

The comparative analysis highlights the complementary nature of these approaches: Newton's method is effective for trajectory optimization, LQR ensures reliable real-time performance, and MPC provides flexibility under constraints. These findings contribute to the development of advanced control solutions relevant to applications in aerial robotics and autonomous systems.

Contents

Introduction **6**

1 System Modeling and Control Theory **8**
1.1 Quadrotor with Suspended Load: System Description 8
1.2 Nonlinear Dynamic Modeling of the Quadrotor System 10
1.3 Discretization of the System Dynamics 11
1.4 Linearization and Jacobian Computation 12
1.5 Fundamentals of Optimal Control Theory 13
1.6 Summary 14

2 Newton's Method for Optimal Control **15**
2.1 Problem Formulation and Reference Trajectory 15
2.1.1 Equilibrium Point Computation 15
2.1.2 Definition of the Reference Curve 16
2.1.3 Force Decomposition 16
2.2 Implementation of Newton's Method 17
2.2.1 Algorithm Overview 17
2.2.2 Descent Direction Calculation 17
2.2.3 Input trajectory Update 17
2.2.4 State Trajectory Update 17
2.2.5 Regularization Techniques 18
2.2.6 Affine LQR Problem 18
2.3 Explicit Computation of $\nabla J(u)$ 18
2.4 Analysis of Algorithm Performance and Convergence 19
2.4.1 Convergence Details and Stopping Criterion 20
2.4.2 Plot Interpretation 20
2.4.3 Significance of Results 21
2.4.4 Conclusion 21

3 Smooth Trajectory Generation **22**
3.1 Reference Trajectory as a Smooth Function 22
3.2 Convergence Analysis of Newton's Method 24
3.2.1 Stopping Criterion 24

3.2.2 Open-Loop versus Closed-Loop Implementation 24
3.2.3 Conclusion 26
3.3 Future Directions 26

4 Trajectory Tracking via LQR 27
4.1 LQR-Based Trajectory Tracking 27
4.1.1 System Lineraization 27
4.1.2 LQR Controller Design 28
4.1.3 Disturbance Rejection and Trajectory Tracking 28
4.1.4 Simulation Results 28
4.2 Conclusion 32

5 Model Predictive Control of a Quadrotor 33
5.1 MPC Algorithm: Dynamic Optimization in a Prediction Horizon 33
5.1.1 Predictive Control in a Rolling Horizon Framework . . 33
5.1.2 Optimization at Each Step 33
5.1.3 Constraints 34
5.1.4 Application of the Control Input and Receding Horizon Strategy 34
5.1.5 Iterative Nature and Computational Considerations . 34
5.2 MPC Algorithm Steps 35
5.3 Simulation Results 35
5.3.1 Significance of Results 38
5.4 Summary 38
5.5 Conclusion 38
5.6 Future Research Directions 38

Conclusions 40

Introduction

Optimal control theory provides a rigorous mathematical framework for designing control policies that drive dynamical systems toward desired behaviors while minimizing cost functions. Its application to aerial robotics is of particular interest, where high maneuverability must be achieved without compromising stability or energy efficiency. Among these systems, quadrotors represent a benchmark platform due to their underactuated nature and strong nonlinear dynamics. The problem complexity increases significantly when a suspended load is attached to the vehicle, as the coupled motion between the quadrotor and the pendulum introduces additional degrees of freedom, oscillatory behavior, and constraints on feasible trajectories.

The central goal of this project is the development and comparison of advanced optimal control strategies for a quadrotor with a suspended load. Specifically, the work addresses trajectory generation and trajectory tracking problems through three complementary approaches:

Newton's method

Newton's method for optimal control, which formulates the problem as a nonlinear optimization and iteratively refines state-input trajectories to minimize the cost function. Both step-based and smooth reference curves were considered to evaluate convergence and computational performance

Linear Quadratic Regulator (LQR)

Linear Quadratic Regulator (LQR) design, obtained by linearizing the nonlinear dynamics along the computed optimal trajectories. This feedback strategy enables effective disturbance rejection while maintaining the system close to the reference path.

Model Predictive Control (MPC)

Model Predictive Control (MPC), which extends the analysis by embedding system constraints within a receding horizon framework. The predictive na-

ture of MPC allows the controller to adapt online to variations in initial conditions and external perturbations, at the expense of higher computational demand.

The significance of this study lies in the integration of trajectory optimization and feedback control within a unified framework. By combining Newton's method with LQR and MPC, the project provides insights into the trade-offs between computational cost, robustness, and accuracy in controlling nonlinear aerial systems. Beyond theoretical formulation, each method was validated through simulation, demonstrating the feasibility of stabilizing the quadrotor-load system and highlighting their respective strengths and limitations.

Ultimately, this project contributes to the broader field of autonomous aerial robotics by advancing the understanding of how optimal control techniques can be leveraged for systems with complex, coupled dynamics. The results are directly relevant for real-world applications such as aerial transportation of payloads, search-and-rescue operations, and cooperative multi-drone missions, where precision and robustness are more important.

Chapter 1

System Modeling and Control Theory

1.1 Quadrotor with Suspended Load: System Description

The system under investigation is a planar quadrotor with a suspended load, which is a representative model for aerial vehicles performing cargo transport The configuration consists of a quadrotor body actuated by two propellers and a point-mass payload attached via a rigid, massless pendulum of length L. This setup introduces underactuation and strong nonlinear couplings between the quadrotor motion and the load dynamics, thereby increasing the complexity of the control problem.

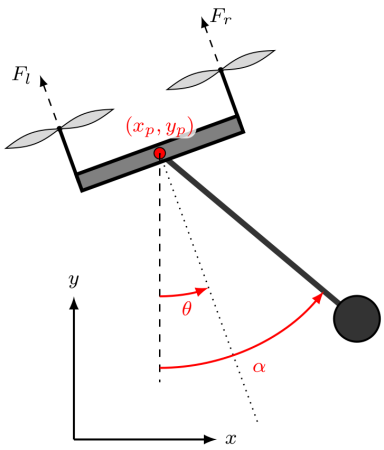


Figure 1.1: Quadrotor Model

A schematic of the system is illustrated in Figure 1.1. The quadrotor operates in a two-dimensional plane, where the pendulum swings freely under the influence of gravity and thrust.

Variables	**Description**
x_p, y_p	Position of the Quadrotor Center of Mass
θ	Roll angle of the quadrotor
α	Angle of pendulum w.r.t vertical axis
v_x, v_y	The Velocity of the center of mass
ω_α	Angular Velocity of the Pendulum
ω_θ	Angular Velocity of the Quadrotor
F_l	Thrust generated by left propeller
F_r	Thrust generated by right propeller

Table 1.1: State Variables

The State Space is defined as

$$x = [x_p, y_p, \alpha, \theta, v_x, v_y, \omega_\alpha, \omega_\theta]^\top \tag{1.1}$$

The control inputs are defined as

$$u = [F_s, F_d]^\top \tag{1.2}$$

where:

- $F_s = F_l + F_r$ - Total thrusts
- $F_d = F_l - F_r$ - Differential thrust

This formulation enables decoupling of altitude control (via F_s) and attitude control (via F_d).

Parameters	**Description**	**Value**
M	Mass of the Quadrotor	0.028
m	Mass of suspended payload	0.040
J	Inertia of the Quadrotor	0.001
g	Gravity	9.81
L	Length of pendulum	0.2
l	Length Between CoM and Propeller	0.05

Table 1.2: System Variables

1.2 Nonlinear Dynamic Modeling of the Quadrotor System

The quadrotor with suspended load is modeled as per given constrains of the project. The suspended pendulum introduces additional dynamic couplings between translational and rotational states, which make the system nonlinear and underactuated.

The continuous-time dynamics of the system are given by:

$$\dot{x}_p = v_x, \tag{1.3}$$

$$\dot{y}_p = v_y, \tag{1.4}$$

$$\dot{\alpha} = \omega_\alpha, \tag{1.5}$$

$$\dot{\theta} = \omega_\theta, \tag{1.6}$$

$$\begin{aligned}\dot{v}_x = &\frac{1}{(M+m)} mL\omega_\alpha^2 \sin(\alpha) \\ &- F_s \left(\sin(\theta) - \frac{m}{M}\sin(\alpha-\theta)\cos(\alpha)\right),\end{aligned} \tag{1.7}$$

$$\begin{aligned}\dot{v}_y = &\frac{1}{(M+m)}(-mL\omega_\alpha^2 \cos(\alpha) \\ &+ F_s \left(\cos(\theta) + \frac{m}{M}\sin(\alpha-\theta)\sin(\alpha)\right) - (M+m)g,\end{aligned} \tag{1.8}$$

$$\dot{\omega}_\alpha = \frac{-F_s \sin(\alpha-\theta)}{ML}, \tag{1.9}$$

$$\dot{\omega}_\theta = \frac{\ell F_d}{J}. \tag{1.10}$$

Interpretation of the Model

- **Translational dynamics:** The horizontal acceleration $\dot{v}_x$ is influenced not only by the roll angle θ but also by the pendulum's oscillations. The vertical acceleration $\dot{v}_y$ balances gravitational force, pendulum swing, and the total thrust F_s.

- **Pendulum dynamics:** The suspended mass swings due to the difference in motion between the quadrotor body and the load. The term $-F_s \sin(\alpha - \theta)$ couples the quadrotor roll and the pendulum angular motion.

- **Rotational dynamics:** The angular acceleration of the quadrotor, $\dot{\omega}_\theta$, is directly proportional to the differential thrust F_d, scaled by the lever arm ℓ.

This nonlinear model serves as the foundation for trajectory generation and controller design. While the full nonlinear dynamics are used for simulation and analysis, many control strategies (e.g., LQR, MPC) require lin-

earized approximations around an equilibrium point or reference trajectory. This motivates the next step: discretization and linearization.

1.3 Discretization of the System Dynamics

To implement control algorithms in a digital computation environment, the continuous-time nonlinear dynamics must be discretized. The system is expressed in compact form as

$$\dot{x}(t) = f(x(t), u(t)), \tag{1.11}$$

where $x \in \mathbb{R}^8$ is the state vector and $u \in \mathbb{R}^2$ is the control input.

For numerical implementation, the forward Euler method is adopted:

$$x_{k+1} = x_k + \Delta t\, f(x_k, u_k), \tag{1.12}$$

where Δt is the discretization step size.

Expanding the equations yields the discrete-time model:

$$x_p^{(k+1)} = x_p^{(k)} + \Delta t\, v_x^{(k)}, \tag{1.13}$$
$$y_p^{(k+1)} = y_p^{(k)} + \Delta t\, v_y^{(k)}, \tag{1.14}$$
$$\alpha^{(k+1)} = \alpha^{(k)} + \Delta t\, \omega_\alpha^{(k)}, \tag{1.15}$$
$$\theta^{(k+1)} = \theta^{(k)} + \Delta t\, \omega_\theta^{(k)}, \tag{1.16}$$
$$v_x^{(k+1)} = v_x^{(k)} + \Delta t\, \Phi_x(x^{(k)}, u^{(k)}), \tag{1.17}$$
$$v_y^{(k+1)} = v_y^{(k)} + \Delta t\, \Phi_y(x^{(k)}, u^{(k)}), \tag{1.18}$$
$$\omega_\alpha^{(k+1)} = \omega_\alpha^{(k)} + \Delta t\, \Phi_\alpha(x^{(k)}, u^{(k)}), \tag{1.19}$$
$$\omega_\theta^{(k+1)} = \omega_\theta^{(k)} + \Delta t\, \Phi_\theta(u^{(k)}). \tag{1.20}$$

Here the nonlinear terms are defined as:

$$\begin{aligned}\Phi_x(x,u) = \frac{1}{M+m}\Big[&mL\omega_\alpha^2 \sin(\alpha) \\ &- F_s\Big(\sin(\theta) - \tfrac{m}{M}\sin(\alpha-\theta)\cos(\alpha)\Big)\Big],\end{aligned} \tag{1.21}$$

$$\begin{aligned}\Phi_y(x,u) = \frac{1}{M+m}\Big[&-mL\omega_\alpha^2 \cos(\alpha) \\ &+ F_s\Big(\cos(\theta) + \tfrac{m}{M}\sin(\alpha-\theta)\sin(\alpha)\Big) - (M+m)g\Big],\end{aligned} \tag{1.22}$$

$$\Phi_\alpha(x,u) = \frac{-F_s \sin(\alpha-\theta)}{ML}, \tag{1.23}$$

$$\Phi_\theta(u) = \frac{\ell F_d}{J}. \tag{1.24}$$

1.4 Linearization and Jacobian Computation

Many control techniques such as LQR and MPC require a linear model of the system. Since the quadrotor with suspended load is governed by nonlinear dynamics, it must be linearized around an operating point or a reference trajectory.

The nonlinear discrete dynamics can be written as

$$x_{k+1} = f_d(x_k, u_k), \tag{1.25}$$

where $f_d(\cdot)$ represents the discretized update function.

To obtain a local linear approximation, we expand f_d using a first-order Taylor series around the point (x^*, u^*):

$$\Delta x_{k+1} = A\,\Delta x_k + B\,\Delta u_k, \tag{1.26}$$

where

$$\Delta x_k = x_k - x^*, \qquad \Delta u_k = u_k - u^*. \tag{1.27}$$

The system matrices A and B are the Jacobians of the discrete dynamics:

$$A = \frac{\partial f_d}{\partial x}\Big|_{(x^*,u^*)}, \qquad B = \frac{\partial f_d}{\partial u}\Big|_{(x^*,u^*)}. \tag{1.28}$$

And the matrices were manually calculated, and the values are:

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & \Delta t \\ 0 & 0 & A_{5,3} & A_{5,4} & 1 & 0 & A_{5,7} & 0 \\ 0 & 0 & A_{6,3} & A_{6,4} & 0 & 1 & A_{6,7} & 0 \\ 0 & 0 & A_{7,3} & A_{7,4} & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ B_{5,1} & 0 \\ B_{6,1} & 0 \\ B_{7,1} & 0 \\ 0 & B_{8,2} \end{bmatrix}.$$

To complement the analytical derivations, the nonlinear dynamics and Jacobian computations were implemented in Python within the file `Quadrotor.py`. The class-based design encapsulates the system parameters, simulation settings, and model functions in a modular manner.

The method `Quadrotor_Dynamics` applies Euler forward integration with a step size of $\Delta t = 10^{-3}$ s to update the state vector given current states and control inputs. This provides a discrete-time realization of the nonlinear model introduced in Section 1.2, consistent with the problem setup required in Task 0 of the project.

The class also includes a `Jacobian` function, which explicitly computes the matrices A and B from the partial derivatives of the discretized dynamics. This function mirrors the manual derivations in Section 1.4.

Note:

$$A_{5,3} = \Delta t \, \frac{1}{M+m} \Big[mL\,\omega_\alpha^2 \cos\alpha \; + \; F_s \, \frac{m}{M} \, \cos(2\alpha - \theta) \Big] \,, \tag{1.29}$$

$$A_{5,4} = \Delta t \left(- \frac{1}{M+m} \Big[F_s \big(\cos\theta + \tfrac{m}{M} \cos(\alpha - \theta) \cos\alpha \big) \Big] \right), \tag{1.30}$$

$$A_{5,7} = \Delta t \, \frac{1}{M+m} \left[2\, mL\, \omega_\alpha \sin\alpha \right], \tag{1.31}$$

$$A_{6,3} = \Delta t \, \frac{1}{M+m} \Big[mL\,\omega_\alpha^2 \sin\alpha \; + \; F_s \, \frac{m}{M} \, \sin(2\alpha - \theta) \Big] \,, \tag{1.32}$$

$$A_{6,4} = \Delta t \left(- \frac{1}{M+m} \Big[F_s \big(\sin\theta + \tfrac{m}{M} \cos(\alpha - \theta) \sin\alpha \big) \Big] \right), \tag{1.33}$$

$$A_{6,7} = \Delta t \left(- \frac{1}{M+m} \Big[2\, mL\, \omega_\alpha \cos\alpha \Big] \right), \tag{1.34}$$

$$A_{7,3} = \Delta t \, \left(- \frac{F_s}{ML} \cos(\alpha - \theta) \right), \tag{1.35}$$

$$A_{7,4} = \Delta t \, \left(\frac{F_s}{ML} \cos(\alpha - \theta) \right), \tag{1.36}$$

$$B_{5,1} = \Delta t \, \left(- \frac{1}{M+m} \Big[\sin\theta - \tfrac{m}{M} \sin(\alpha - \theta) \cos\alpha \Big] \right), \tag{1.37}$$

$$B_{6,1} = \Delta t \, \left(\frac{1}{M+m} \Big[\cos\theta + \tfrac{m}{M} \sin(\alpha - \theta) \sin\alpha \Big] \right), \tag{1.38}$$

$$B_{7,1} = \Delta t \, \left(- \frac{1}{ML} \sin(\alpha - \theta) \right), \tag{1.39}$$

$$B_{8,2} = \Delta t \, \left(\frac{\ell}{J} \right). \tag{1.40}$$

1.5 Fundamentals of Optimal Control Theory

Optimal control theory is a mathematical framework concerned with determining a control policy for a dynamical system such that a prescribed performance objective is optimized. In the context of quadrotor systems, this objective typically involves maneuvering the vehicle along a desired trajectory while minimizing energy expenditure, ensuring stability, and respecting system dynamics.

The finite-horizon nonlinear optimal control problem is formulated as:

$$\min_{\substack{x_1, \ldots, x_T, \\ u_0, \ldots, u_{T-1}}} \sum_{t=0}^{T-1} \left(\tfrac{1}{2} \|x_t - x_{des,t}\|_{Q_t}^2 + \tfrac{1}{2} \|u_t - u_{des,t}\|_{R_t}^2 \right) + \tfrac{1}{2} \|x_T - x_{des,T}\|_{P_f}^2 \tag{1.41}$$

subject to

$$x_{t+1} = f(x_t, u_t), \qquad t = 0, \ldots, T-1, \tag{1.42}$$

$$x_0 = x_{\text{init}}. \tag{1.43}$$

Here, Q_t, R_t and P_f are user-defined weighting matrices penalizing state deviations, control effort, and terminal error, respectively. The sequences $x_{des,t}$ and $u_{des,t}$ represent desired reference trajectories that need not necessarily satisfy the dynamics exactly.

In the present work, the nonlinear dynamics $f(\cdot)$ of the quadrotor with suspended load (see Section 1.2) are used in this formulation. Discretization (Section 1.3) allows implementation in a digital simulation environment, while linearization (Section 1.4) provides locally valid models that enable the application of Newton's method, Linear Quadratic Regulation (LQR), and Model Predictive Control (MPC).

This framework bridges system modeling with algorithmic design, laying the theoretical foundation for the trajectory generation and control strategies developed in the subsequent chapters.

1.6 Summary

In this chapter, we have established the theoretical and modeling framework required for the optimal control of a quadrotor with a suspended load. We began with a detailed description of the system and its states, followed by the derivation of the nonlinear dynamic equations governing the translational, rotational, and pendulum motions. These continuous dynamics were then discretized using Euler integration to enable implementation in a digital computation environment, consistent with the requirements of modern control algorithms.

Subsequently, the Jacobian matrices were derived to obtain linearized models around generic operating points. These approximations are essential for trajectory optimization and for the design of feedback controllers such as LQR and MPC. Alongside the manual derivation, the implementation of the dynamics and Jacobian computation in Python provided a practical means to simulate and validate the system behavior. Finally, the fundamentals of optimal control theory were presented, forming the bridge between system modeling and advanced control design.

Together, these steps form a comprehensive foundation for the development of optimal control strategies. The next chapters build upon this groundwork by addressing trajectory generation through Newton's method, followed by trajectory tracking using LQR and MPC.

Chapter 2

Newton's Method for Optimal Control

2.1 Problem Formulation and Reference Trajectory

The first step in applying Newton's method for optimal control is the formulation of the reference trajectory. In the context of the quadrotor with suspended load, this trajectory represents a desired evolution of the system states that connects two feasible equilibria of the dynamics. The goal of the optimization procedure is not only to generate an admissible trajectory that satisfies the nonlinear dynamics, but also to minimize the associated quadratic cost function with respect to deviations from the reference.

2.1.1 Equilibrium Point Computation

Equilibrium points of the quadrotor–pendulum system are defined as states in which both the translational and rotational accelerations vanish under constant control inputs. Formally, an equilibrium (x_e, u_e) satisfies:

$$f(x_e, u_e) = 0, \tag{2.1}$$

where $f(\cdot)$ represents the continuous-time system dynamics. For the planar quadrotor, this requires balancing gravitational forces with the supporting thrust F_s, while ensuring that the differential thrust F_d is zero to avoid roll acceleration. Typical choices include:

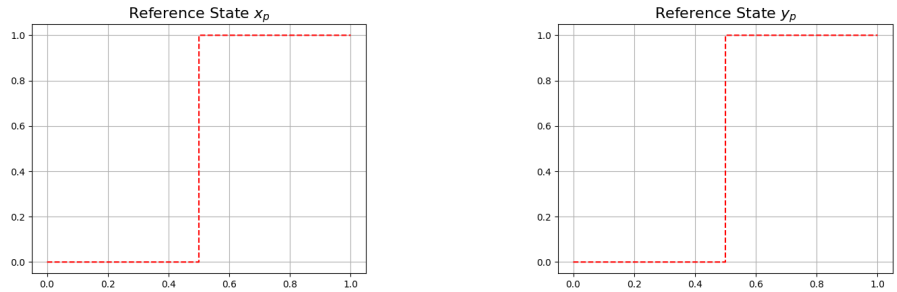
- A hovering equilibrium at $(x_p, y_p) = (0, 0)$ with $F_s = (M+m)g$, $F_d = 0$.
- A shifted equilibrium at $(x_p, y_p) = (0, 0)$ with the same force balance, corresponding to the target state.

The determination of such equilibria can be automated by root-finding routines applied to the system equations. These equilibria serve as the initial and terminal conditions for the optimal control problem.

2.1.2 Definition of the Reference Curve

Once the equilibrium points are known, a reference trajectory must be defined between them. The reference is not necessarily dynamically feasible but provides a geometric guideline for the optimizer. For Task 1, the reference is modeled as a **step function** in position: the quadrotor is assumed to remain at the initial equilibrium for a prescribed duration, transition towards the target, and then stabilize at the final equilibrium.

This formulation is consistent with classical optimal control setups, where the desired path simplifies the design of cost matrices while leaving the optimizer to enforce feasibility through the system dynamics.



(a) State x_p (b) State y_p

Figure 2.1: Step reference trajectory in the (x,y) plane used as baseline for Newton's method.

2.1.3 Force Decomposition

The control inputs of the system are expressed as:

- **Supporting force:** $F_s = F_l + F_r$, the total vertical thrust. At equilibrium, this exactly cancels the gravitational pull (M + m)g. Adjustments in F_s enable altitude changes
- **Differential force:** $F_d = F_r - F_l$, responsible for roll torque and lateral displacement. Setting $F_d = 0$ eliminates roll dynamics, corresponding to a purely vertical maneuver.

In defining the reference curve, we consider the nominal case $F_d = 0$, $F_s = (M + m)g$, and all velocities and angular states equal to zero. This baseline reference provides the optimizer with a simple target to refine, while allowing Newton's method to generate dynamically consistent trajectories that respect the nonlinear quadrotor–pendulum model.

2.2 Implementation of Newton's Method

Newton's method is applied to solve the optimal control problem by iteratively refining the state–input trajectory in order to minimize the cost functional. The algorithm uses local linearizations of the dynamics and second-order approximations of the cost to compute descent directions that progressively improve feasibility and optimality.

2.2.1 Algorithm Overview

The procedure starts with an initial guess (x^0, u^0) and updates it iteratively until convergence. At each iteration, gradients and Hessians of the dynamics and cost are computed, leading to a quadratic subproblem whose solution provides the update direction.

2.2.2 Descent Direction Calculation

The gradients and Hessians are computed as follows:

Evaluate $\nabla_1 f_t(x_k^t, u_k^t), \nabla_2 f_t(x_k^t, u_k^t), \nabla_1 \ell_t(x_k^t, u_k^t), \nabla_2 \ell_t(x_k^t, u_k^t), \nabla \ell_T(x_k^T)$.
Co-state Equation: $\lambda_k^T = \nabla \ell_T(x_k^T)$.
Descent Direction: Solve the minimization problem:

$$\min_{\Delta x, \Delta u} \sum_{t=0}^{T-1} \begin{bmatrix} \nabla_1 \ell_t(x_k^t, u_k^t) \\ \nabla_2 \ell_t(x_k^t, u_k^t) \end{bmatrix}^\top \begin{bmatrix} \Delta x_t \\ \Delta u_t \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \Delta x_t \\ \Delta u_t \end{bmatrix}^\top \begin{bmatrix} Q_k^t & S_k^{t\top} \\ S_k^t & R_k^t \end{bmatrix} \begin{bmatrix} \Delta x_t \\ \Delta u_t \end{bmatrix}$$
$$+ \nabla \ell_T(x_k^T)^\top \Delta x_T + \frac{1}{2} \Delta x_T^\top Q_k^T \Delta x_T \tag{2.2}$$
$$\text{s.t. } \Delta x_{t+1} = \nabla_1 f_t(x_k^t, u_k^t)^\top \Delta x_t + \nabla_2 f_t(x_k^t, u_k^t)^\top \Delta u_t, \Delta x_0 = 0$$

2.2.3 Input trajectory Update

Compute new state-input trajectory (implementing step-size selection rule, e.g., Armijo):

$$OpenLoop : u_{k+1}^t = u_k^t + \gamma_k \Delta u_k^t. \tag{2.3}$$
$$ClosedLoop : u_{k+1}^t = u_k^t + u_k^t(x_{k+1}^t - x_k^t) + \gamma_k \sigma_t^K. \tag{2.4}$$

2.2.4 State Trajectory Update

The state trajectory is updated by forward integrating with $x_{k+1}^0 = x_{\text{init}}$ and $x_{k+1}^{t+1} = f(x_{k+1}^t, u_{k+1}^t)$.

2.2.5 Regularization Techniques

To handle non-positive definite matrices, the following regularization is applied:

$$\begin{aligned}
\tilde{Q}_k^t &:= \nabla_{2,11}\ell_t(x_k^t, u_k^t),\\
\tilde{R}_k^t &:= \nabla_{2,22}\ell_t(x_k^t, u_k^t),\\
\tilde{S}_k^t &:= \nabla_{2,12}\ell_t(x_k^t, u_k^t),\\
\tilde{Q}_k^T &:= \nabla_{2,11}\ell_T(x_k^T).
\end{aligned}$$

2.2.6 Affine LQR Problem

The affine LQR problem is solved with the following optimal solution:

$$\Delta u_t^* = K_t^* \Delta x_t^* + \sigma_t^*, \tag{2.5}$$

$$\Delta x_{t+1}^* = A_t \Delta x_t^* + B_t \Delta u_t^*, \tag{2.6}$$

$$\text{for } t = 0, \ldots, T-1,$$

where

$$\begin{aligned}
K_t^* &= -(R_t + B_t^\top P_{t+1} B_t)^{-1}(S_t + B_t^\top P_{t+1} A_t),\\
\sigma_t^* &= -(R_t + B_t^\top P_{t+1} B_t)^{-1}\left(r_t + B_t^\top p_{t+1} + B_t^\top P_{t+1} c_t\right),\\
p_t &= q_t + A_t^\top p_{t+1} + A_t^\top P_{t+1} c_t + K_t^{*\top}\left(R_t + B_t^\top P_{t+1} B_t\right)\sigma_t^*,\\
P_t &= Q_t + A_t^\top P_{t+1} A_t - K_t^{*\top}(R_t + B_t^\top P_{t+1} B_t) K_t^*,
\end{aligned}$$

with $p_T = q_T$ and $P_T = Q_T$.

2.3 Explicit Computation of $\nabla J(u)$

To compute $\nabla J(u)$ explicitly, consider the Lagrangian $L(x, u, \lambda)$ defined as:

$$L(x, u, \lambda) = \sum_{t=0}^{T-1} \ell_t(x_t, u_t) + \ell_T(x_T) + \sum_{t=0}^{T-1} \left(f(x_t, u_t) - x_{t+1}\right)^\top \lambda_{t+1}. \tag{2.7}$$

Given a trajectory (x, u), such that $x_{t+1} = f_t(x_t, u_t)$ for $t = 0, 1, \ldots, T-1$ and a given x_0, the gradient at each time step t is computed as follows:

$$\begin{aligned}
\lambda_t &= A_t^\top \lambda_{t+1} + a_t, \quad \text{for } t = T-1, \ldots, 1, \text{ with } \lambda_T = \nabla \ell_T(x_T),\\
(\nabla J(u))_t &= B_t^\top \lambda_{t+1} + b_t, \quad \text{for } t = T-1, \ldots, 0,
\end{aligned}$$

where we denote:

$$
\begin{aligned}
A_t^\top &= \nabla_1 f(x_t, u_t), \\
B_t^\top &= \nabla_2 f(x_t, u_t), \\
a_t &= \nabla_1 \ell_t(x_t, u_t), \\
b_t &= \nabla_2 \ell_t(x_t, u_t).
\end{aligned}
$$

2.4 Analysis of Algorithm Performance and Convergence

The implementation of Newton's method for optimal control has been successfully executed, converging to an optimal trajectory after few iterations.

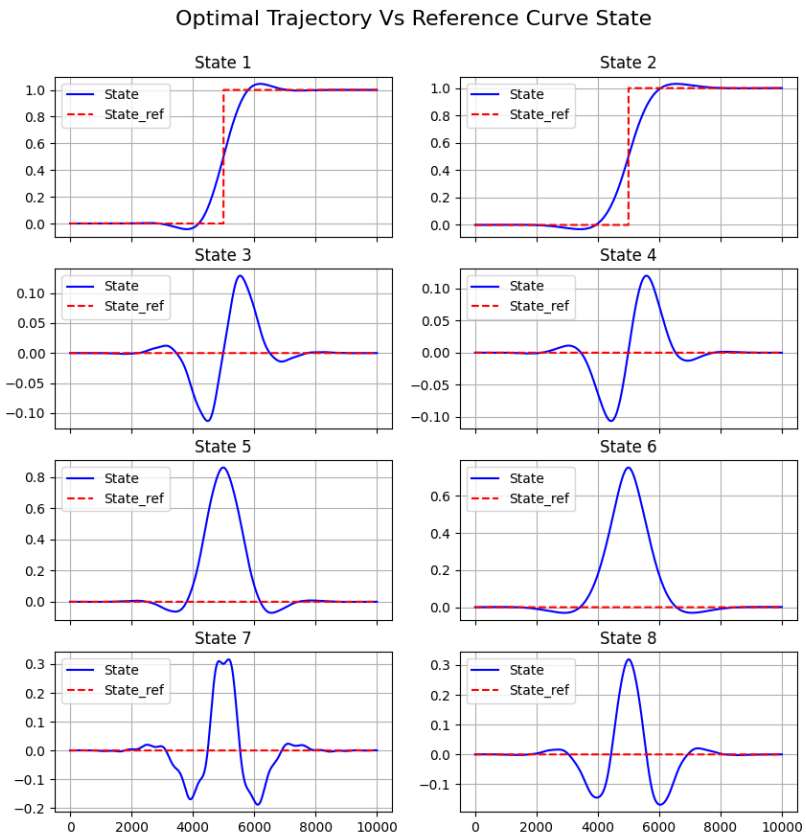
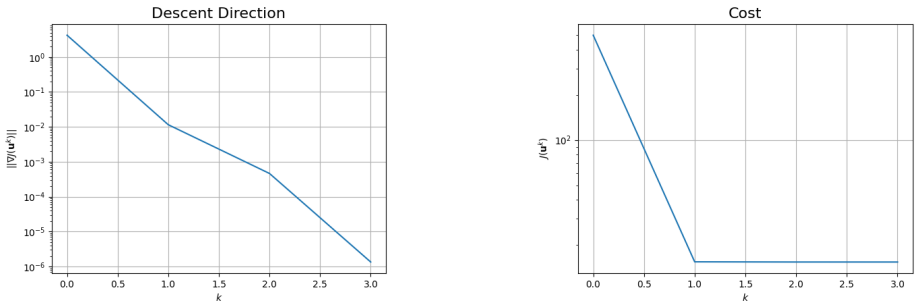


Figure 2.2: Newton's Result "Optimal Vs Reference States



(a) (b)

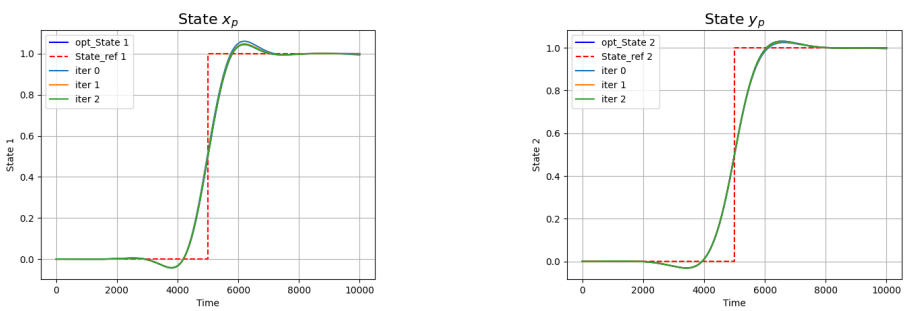
Figure 2.3: (a) Decent Direction and (b) Cost

2.4.1 Convergence Details and Stopping Criterion

The algorithm's convergence was determined using a specific stopping criterion, where the descent magnitude in each iteration was monitored. The termination condition was set such that if `descent[k] <= 1e-6`, the algorithm would cease iterations. In this context, `descent[k]` was computed as the sum of the squared control updates (`du[:,tt]@du[:,tt]`) at each time step. This precise criterion ensured that the algorithm only stopped when a sufficiently accurate solution was reached, avoiding premature termination.

2.4.2 Plot Interpretation

The plot included in Figure 2.4 visually represents the algorithm's progression towards the optimal trajectory. It is evident from the plot that the trajectory corrections significantly decrease as the number of iterations increases, aligning with our expectations of Newton's method. The rapid de-



(a) State x_p (b) State y_p

Figure 2.4: Optimal trajectory, desired curve, and a few intermediate trajectories

crease in trajectory corrections within the initial iterations highlights the algorithm's efficiency.

2.4.3 Significance of Results

The successful convergence of the algorithm within 5 iterations underscores its potential for efficient and precise solutions in optimal control problems. The algorithm's ability to rapidly approximate the optimal trajectory, as demonstrated in the plot, is particularly valuable in scenarios where computational resources or time are constraints.

2.4.4 Conclusion

In conclusion, the results obtained and depicted in the plot affirm the robustness and reliability of Newton's method in solving optimal control problems. The method's convergence behavior and computational efficiency make it a strong candidate for more complex or real-time control scenarios. Future work could explore its application in more diverse control environments or its integration with other computational optimization techniques.

Chapter 3

Smooth Trajectory Generation

The trajectory design problem can be further refined by considering smooth reference functions instead of step-like references. While step trajectories are useful for illustrating feasibility, they introduce discontinuities that may hinder convergence of Newton's method. In contrast, smooth reference curves provide a more realistic and computationally efficient basis for trajectory generation.

3.1 Reference Trajectory as a Smooth Function

The smooth trajectory is constructed using spline interpolation between two equilibrium configurations of the quadrotor system. Specifically, the reference is defined between the initial equilibrium point

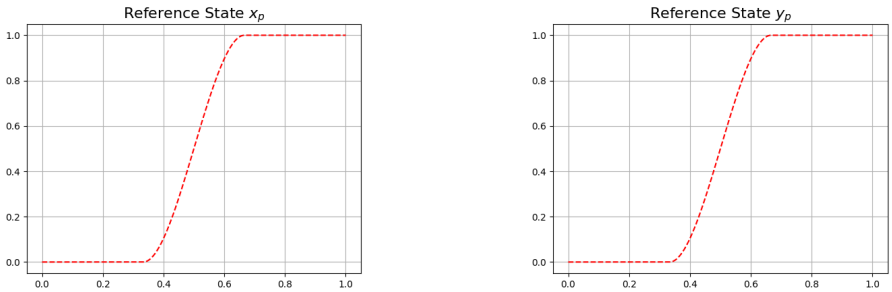
$$(x_{p1}, y_{p1}) = (0, 0)$$

and the target equilibrium point

$$(x_{p2}, y_{p2}) = (1, 1)$$

The corresponding input forces are chosen consistently with the equilibrium conditions:

- The collective force is fixed at $F_s = (M + m)g$ to counteract gravity and maintain altitude.
- The differential force is set to $F_d = 0$ ensuring symmetric thrust between the two propellers.
- All other states (velocities and angular coordinates) are initialized to zero.



(a) State x_p (b) State y_p

Figure 3.1: Smooth reference trajectory in the (x,y) plane Newton's method.

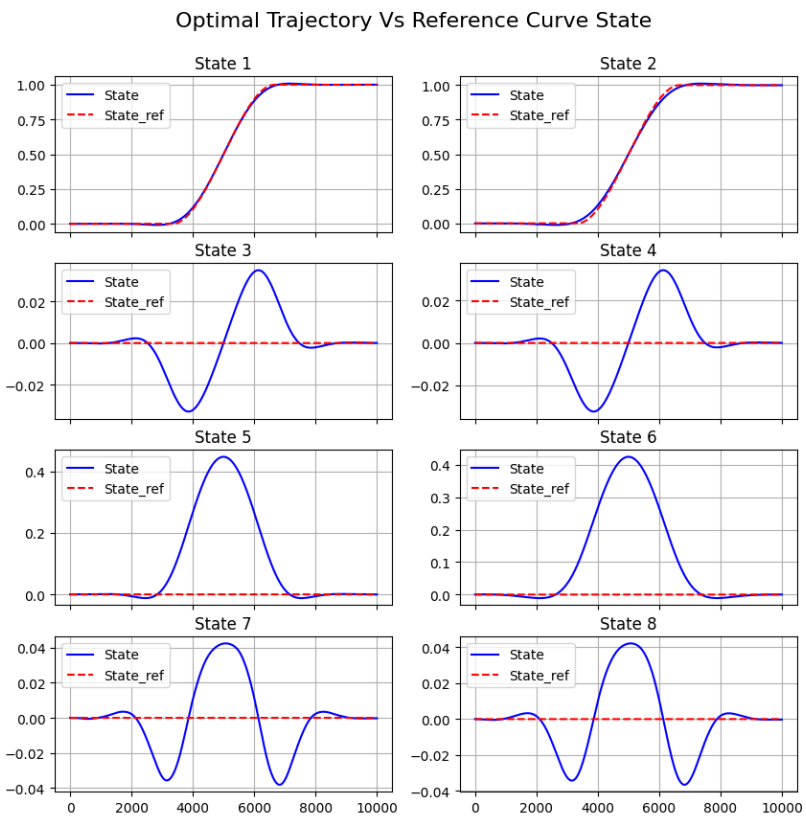


Figure 3.2: Newton's Result "Optimal Vs Reference States

This formulation yields a smooth reference trajectory that is dynamically consistent with the system and avoids abrupt state transitions, thereby

improving the numerical stability of the optimization.

3.2 Convergence Analysis of Newton's Method

Applying Newton's method to this smooth reference trajectory demonstrates faster convergence compared to the step function case discussed in Chapter 2. The smoothness of the reference reduces the nonlinearity introduced by discontinuities, thereby simplifying the optimization landscape.

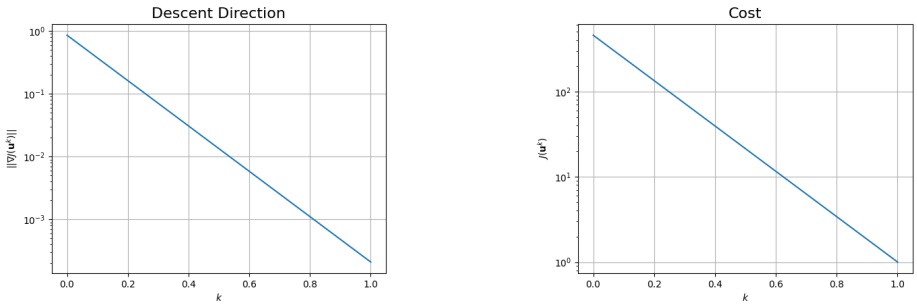
Figure 3.2 illustrates the alignment between the smooth reference curve and the optimized state trajectory obtained from the algorithm. Convergence is achieved within a limited number of iterations, validating the effectiveness of this approach.

3.2.1 Stopping Criterion

The convergence of the algorithm is assessed using a descent-based stopping criterion. Specifically, the iterations terminate when the squared norm of the control updates satisfies:

$$||\Delta u||^2 \leq 10^{-6}$$

Compared to the step-function scenario, the descent direction exhibits a more rapid decay (Figure 3.3a), and the cost function decreases significantly during the first iteration (Figure 3.3b). These results confirm that introducing a smooth reference function enhances both convergence speed and computational efficiency.



(a) (b)

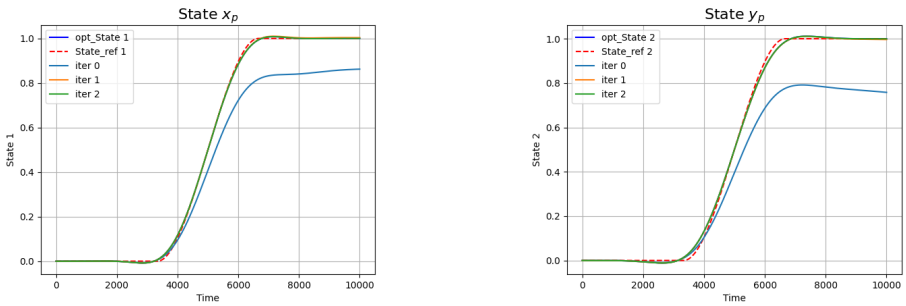
Figure 3.3: (a) Decent Direction and (b) Cost

3.2.2 Open-Loop versus Closed-Loop Implementation

A comparative study of open-loop and closed-loop implementations of Newton's method reveals distinct performance characteristics:

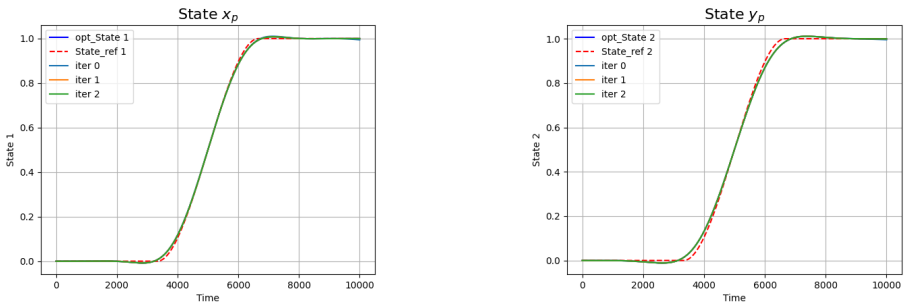
- **Open-loop** convergence is typically achieved within approximately five iterations, with the intermediate trajectories gradually approaching the reference.
- **Closed-loop** convergence occurs in as few as three iterations, with the algorithm producing a trajectory closely aligned with the reference already in the initial iteration.

Figure 3.4 and 3.5 highlight these differences. The closed-loop strategy consistently demonstrates superior convergence accuracy and robustness, making it more suitable for scenarios requiring reliable trajectory generation under real-time constraints.



(a) State x_p (b) State y_p

Figure 3.4: Comparison of intermediate trajectories versus the optimal and desired trajectory using open-loop Newton's method



(a) State x_p (b) State y_p

Figure 3.5: Comparison of intermediate trajectories versus the optimal and desired trajectory using closed-loop Newton's method

3.2.3 Conclusion

The results of this chapter demonstrate that designing smooth reference trajectories substantially improves the performance of Newton's method for optimal control. Both the convergence rate and the computational efficiency are enhanced relative to step-function references. Furthermore, the closed-loop implementation outperforms the open-loop variant, ensuring faster and more accurate convergence to the desired trajectory.

3.3 Future Directions

Future research could extend this methodology to account for external disturbances and model uncertainties. In particular, incorporating robustness into the trajectory generation process—by integrating disturbance models or adaptive update mechanisms—would enhance the applicability of the method to real-world quadrotor operations.

Chapter 4

Trajectory Tracking via LQR

This chapter addresses the design and implementation of a Linear Quadratic Regulator (LQR) for the trajectory tracking problem. The goal is to ensure that the quadrotor with a suspended load follows the optimal trajectory derived in Chapter 2 while exhibiting robustness against disturbances. The LQR approach is particularly suitable due to its balance between computational efficiency and strong theoretical guarantees in stabilizing linearized dynamical systems.

4.1 LQR-Based Trajectory Tracking

The Linear Quadratic Regulator (LQR) is a feedback control method that minimizes a quadratic cost function over a discrete-time horizon. Its primary objective is to regulate deviations of the system's state from the optimal trajectory while simultaneously penalizing excessive control effort. The resulting controller ensures a trade-off between trajectory accuracy and input energy efficiency.

4.1.1 System Lineraization

The quadrotor dynamics are nonlinear, and therefore a linear approximation is required around the optimal trajectory (x_{opt}, u_{opt}) The linearized system is expressed as:

$$\Delta x_{t+1} = A_{opt,t}\Delta x_t + B_{opt,t}\Delta u_t \tag{4.1}$$

where $\Delta x_t = x_t - x_{opt,t}$ and $\Delta u_t = u_t - u_{opt,t}$. The matrices $A_{opt,t}$ and $B_{opt,t}$ are obtained from the Jacobians of the system dynamics evaluated along the optimal trajectory. This linearization enables the application of quadratic optimization techniques inherent in LQR design.

4.1.2 LQR Controller Design

The Linear Quadratic Regulator (LQR) framework formulates the trajectory tracking problem as the minimization of a quadratic cost function subject to the linearized system dynamics. The optimization problem is given by

$$\min_{\Delta u} \sum_{t=0}^{T-1} \left(\Delta x_t^\top Q_{reg} \Delta x_t + \Delta u_t^\top R_{reg} \Delta u_t \right) + \Delta x_T^\top Q_{reg,T} \Delta x_T, \tag{4.2}$$

subject to the system dynamics in eq. 4.1

where Q_{reg}, R_{reg}, and $Q_{reg,T}$ are the state, input, and terminal cost weighting matrices, respectively.

The solution of this problem is obtained via the backward Riccati recursion. The symmetric positive definite matrix P_t and the optimal gain matrix K_t are computed iteratively as follows:

$$\begin{aligned} P_t = {} & Q_{reg,t} + A_{opt,t}^\top P_{t+1} A_{opt,t} \\ & - \left(A_{opt,t}^\top P_{t+1} B_{opt,t} \right) \left(R_{reg,t} + B_{opt,t}^\top P_{t+1} B_{opt,t} \right)^{-1} \left(B_{opt,t}^\top P_{t+1} A_{opt,t} \right) \end{aligned} \tag{4.3}$$

$$K_t = - \left(R_{reg,t} + B_{opt,t}^\top P_{t+1} B_{opt,t} \right)^{-1} \left(B_{opt,t}^\top P_{t+1} A_{opt,t} \right). \tag{4.4}$$

The recursion is initialized with $P_T = Q_{reg,T}$, and the process is repeated for all $t = T-1, \ldots, 0$. The resulting time-varying gain matrices K_t define the optimal feedback law, ensuring regulation of the system states around the optimal trajectory.

4.1.3 Disturbance Rejection and Trajectory Tracking

The closed-loop control law is expressed as:

$$u_t = u_{opt,t} + K_t(x_t - x_o pt, t) \tag{4.5}$$

which actively corrects deviations from the desired trajectory. The LQR controller thus guarantees asymptotic stability of the linearized system and ensures robustness to external disturbances or modeling uncertainties. Disturbance rejection arises from the feedback term, which continuously drives the state back towards the optimal reference trajectory.

4.1.4 Simulation Results

Numerical simulations were performed to evaluate the performance of the LQR controller.

- **Without LQR:** Figure 4.1 illustrates the deviation of the actual trajectory from the optimal trajectory when disturbances are present, but no feedback correction is applied. Significant drift from the reference trajectory is observed.

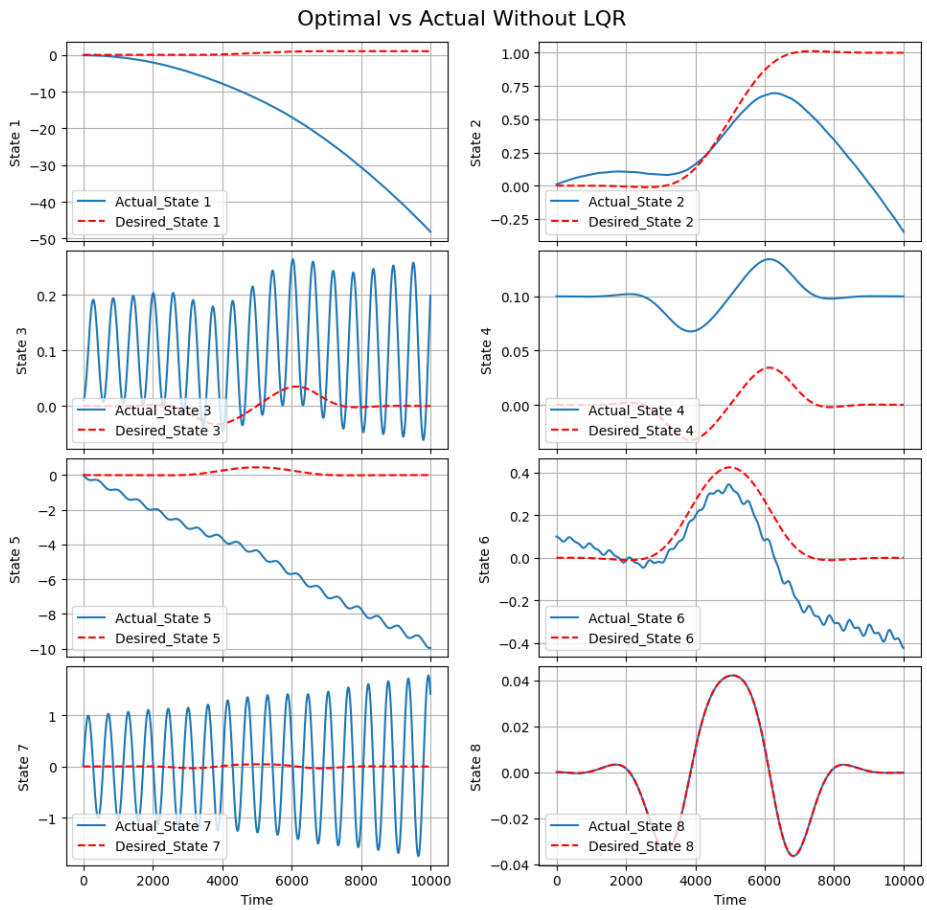


Figure 4.1: Optimal trajectory with the actual trajectory under disturbance, without LQR tracking

- With LQR: Figure 4.2 demonstrates the corrected trajectory under LQR control. The controller effectively mitigates the effects of disturbances, ensuring that the actual trajectory closely follows the reference.

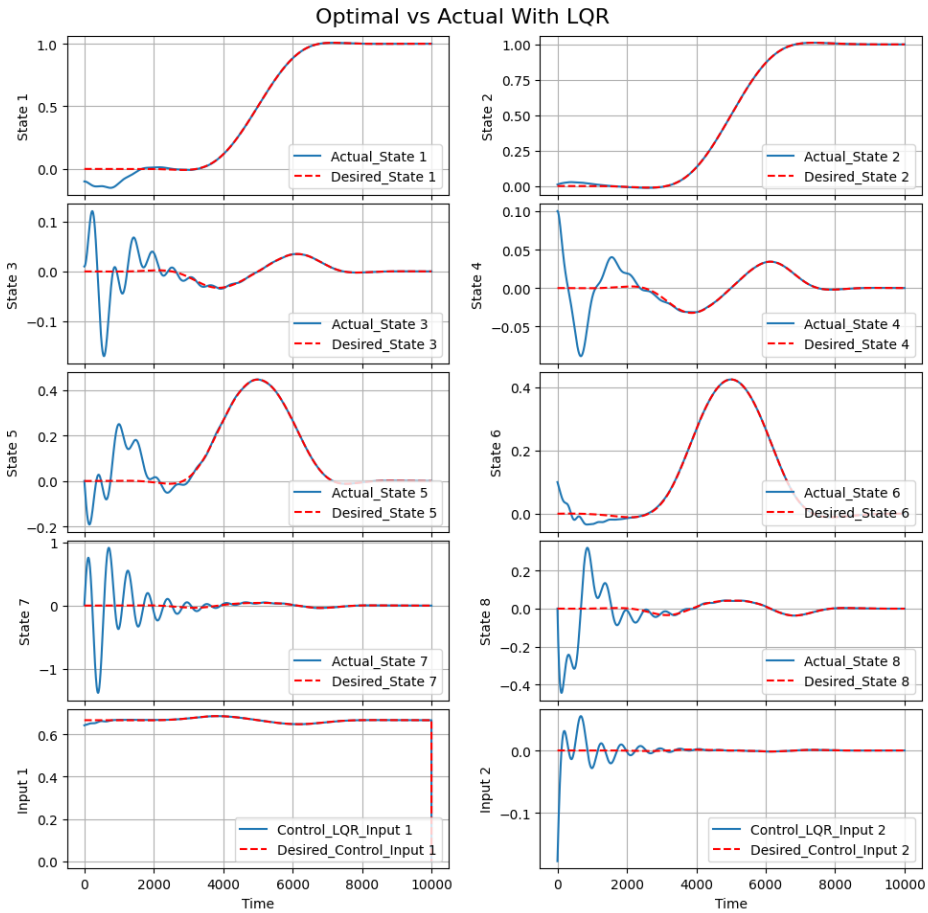


Figure 4.2: Optimal trajectory with the actual trajectory under disturbance, with LQR tracking

- Tracking Error: Figures 4.3 and 4.4 quantify the deviation between the actual and optimal trajectories. The error remains bounded within ± 0.05, and the logarithmic error plot confirms exponential convergence towards the reference trajectory.

These results validate the LQR framework as a reliable strategy for disturbance rejection and trajectory stabilization.

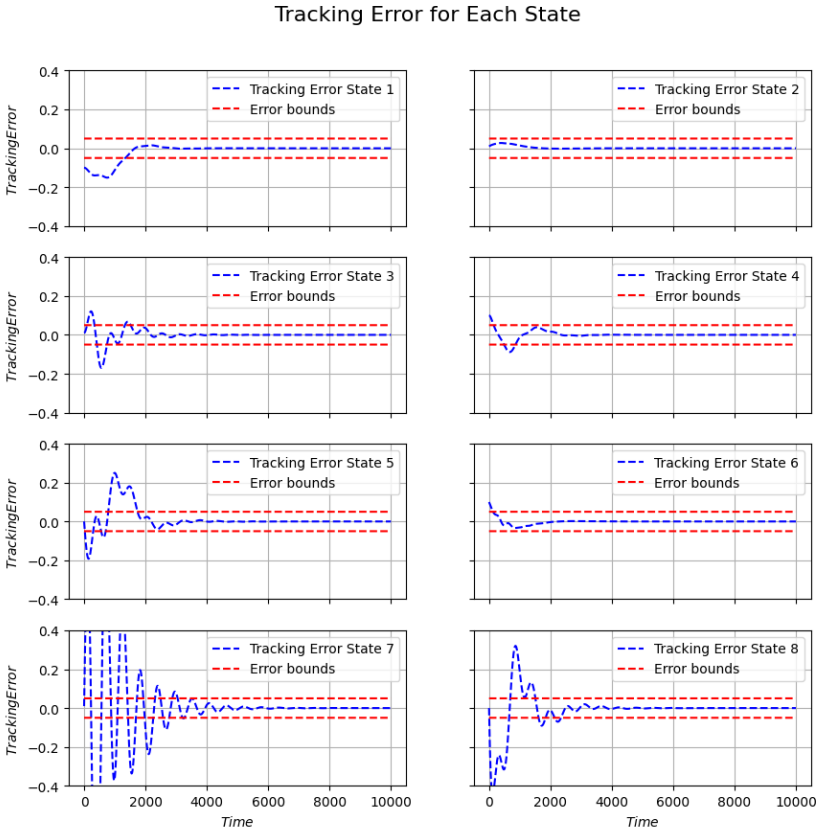


Figure 4.3: Tracking error over time with LQR controller

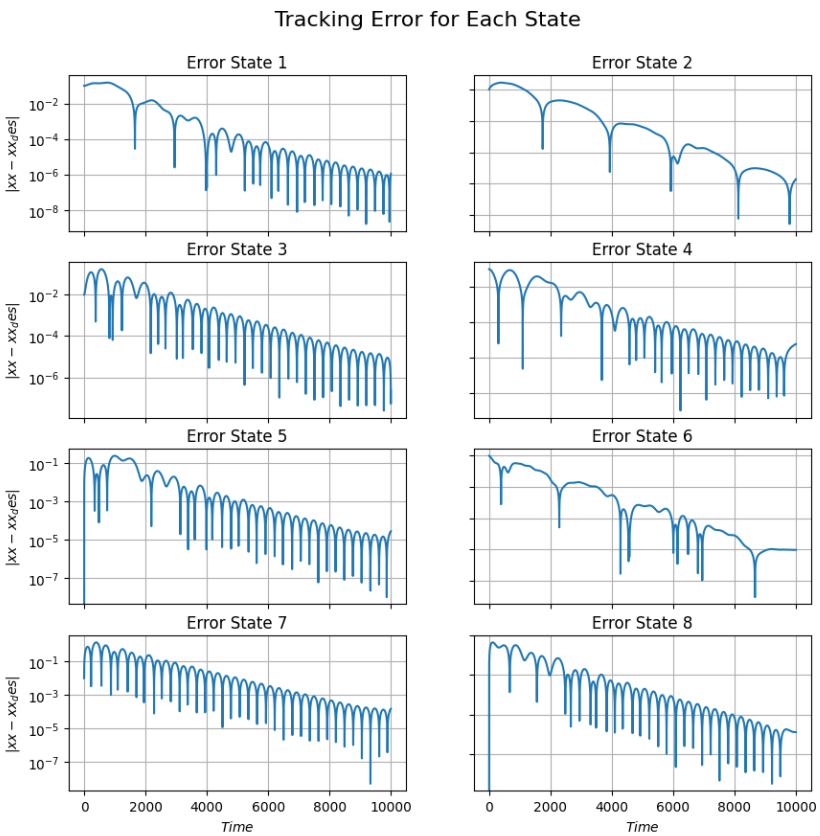


Figure 4.4: Tracking error over time with LQR controller ”Logarithmic scale"

4.2 Conclusion

This chapter has demonstrated the effectiveness of the LQR approach for trajectory tracking of a quadrotor with a suspended load. By linearizing the system around the optimal trajectory and designing a state-feedback controller through the Riccati recursion, the quadrotor achieves robust performance even under external disturbances. The simulation results confirm that the LQR controller significantly reduces trajectory deviations, thereby reinforcing its suitability for real-time implementation in aerial robotics applications.

Chapter 5

Model Predictive Control of a Quadrotor

5.1 MPC Algorithm: Dynamic Optimization in a Prediction Horizon

The essence of Model Predictive Control (MPC) lies in its iterative, predictive, and optimizing nature, particularly evident in the use of a finite prediction horizon. In the following, we present the core algorithmic steps of MPC in the context of quadrotor trajectory tracking.

5.1.1 Predictive Control in a Rolling Horizon Framework

MPC is based on the principle of repeatedly solving an optimization problem over a moving (receding) horizon. At each sampling time t, the controller predicts the evolution of the quadrotor states within a finite prediction horizon T_{pred}. This prediction accounts for both the system dynamics and possible external disturbances. By exploiting these predictions, MPC determines a control sequence that minimizes a prescribed cost function while respecting constraints.

5.1.2 Optimization at Each Step

At each time step t, MPC solves the following optimization problem:

$$\min_{x_t,\ldots,x_{t+T},\, u_t,\ldots,u_{t+T-1}} \sum_{\tau=t}^{t+T-1} \left(x_\tau^\top Q x_\tau + u_\tau^\top R u_\tau \right) + x_{t+T}^\top P x_{t+T} \tag{5.1}$$

subject to the constraints:

$$\begin{aligned} x_{\tau+1} &= f(x_\tau, u_\tau), \quad \tau = t, \ldots, t+T-1, \\ x_t &= x_{\text{meas},t}, \\ u_\tau &\in \mathcal{U}, \quad \tau = t, \ldots, t+T-1. \end{aligned}$$

Here, $x_t, \ldots, x_{t+T}$ and $u_t, \ldots, u_{t+T-1}$ denote the predicted state and input trajectories, respectively. The weighting matrices Q and R penalize deviations from the reference state and excessive control effort, while P is the terminal state weighting matrix obtained from the Algebraic Riccati Equation (ARE). The function $f(\cdot)$ represents the nonlinear quadrotor dynamics, and $x_{\text{meas},t}$ is the measured system state at time t. The set $\mathcal{U}$ denotes the feasible input region, which incorporates actuator limitations.

5.1.3 Constraints

In practice, quadrotor operation must satisfy physical and safety limitations. These are expressed as input and state constraints of the form:

$$u_{\min} \leq u_k \leq u_{\max}, \tag{5.2}$$

$$x_{\min} \leq x_k \leq x_{\max}. \tag{5.3}$$

These conditions ensure that the control inputs remain within the actuator bounds and that the states evolve within admissible operating regions.

5.1.4 Application of the Control Input and Receding Horizon Strategy

Once the optimization problem is solved, only the first control input of the optimal sequence is applied to the quadrotor. At the next sampling instant, the state is remeasured, the horizon shifts forward by one step, and a new optimization problem is solved. This receding horizon strategy enables MPC to continuously adapt to changes in the quadrotor's dynamics, disturbances, or modeling errors.

5.1.5 Iterative Nature and Computational Considerations

The strength of MPC lies in its predictive capability and iterative structure, which combine foresight with adaptability. However, this advantage comes with the requirement of solving an optimization problem at each time step. For systems with fast dynamics, such as quadrotors, this results in significant computational demands. Efficient solvers and real-time implementation strategies are therefore essential for practical deployment of MPC.

5.2 MPC Algorithm Steps

The MPC control loop at each sampling instant can be summarized as follows:

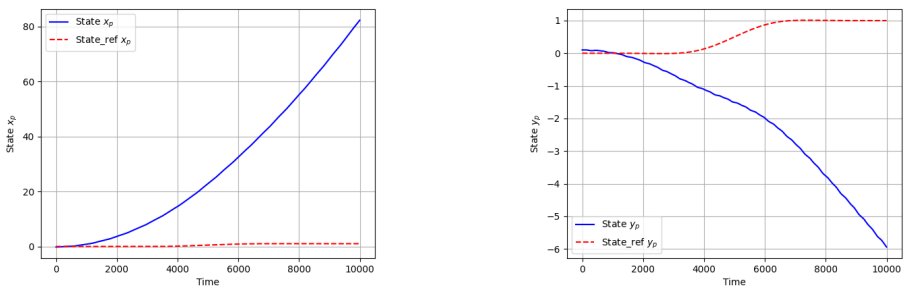
1. Measure the current state.
2. Solve the finite-horizon optimization problem.
3. Apply the first control input from the computed sequence.
4. Update the state prediction with the new system state.
5. Repeat at the next time step.

5.3 Simulation Results

The MPC controller was tested for trajectory tracking and disturbance rejection. Simulation results are provided for key states, particularly x_p and y_p:

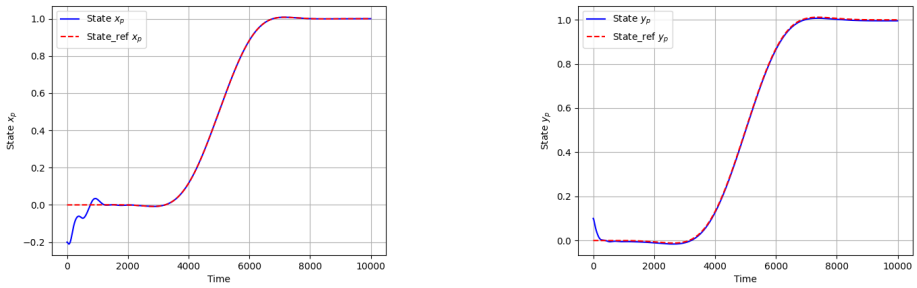
- **Without MPC (Figures 5.1):** Disturbances lead to significant deviation from the desired trajectory.
- **With MPC (Figures 5.2):** The controller substantially improves disturbance rejection and ensures close tracking of the reference trajectory.

Tracking error plots (Figures 5.3 and 5.4) confirm that MPC achieves precise convergence to the desired trajectory, with errors decreasing significantly over time.



(a) State x_p (b) State y_p

Figure 5.1: Trajectory Tracking Performance without MPC Implementation in Disturbance Handling



(a) State x_p (b) State y_p

Figure 5.2: Trajectory Tracking Performance with MPC Implementation in Disturbance Handling

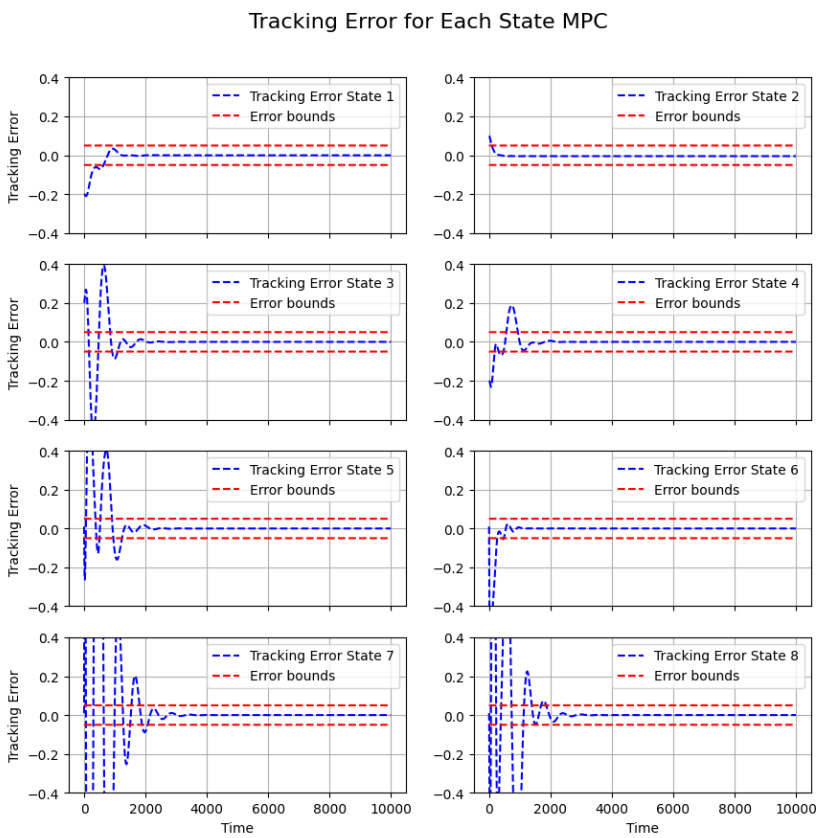


Figure 5.3: Tracking error over time with MPC controller

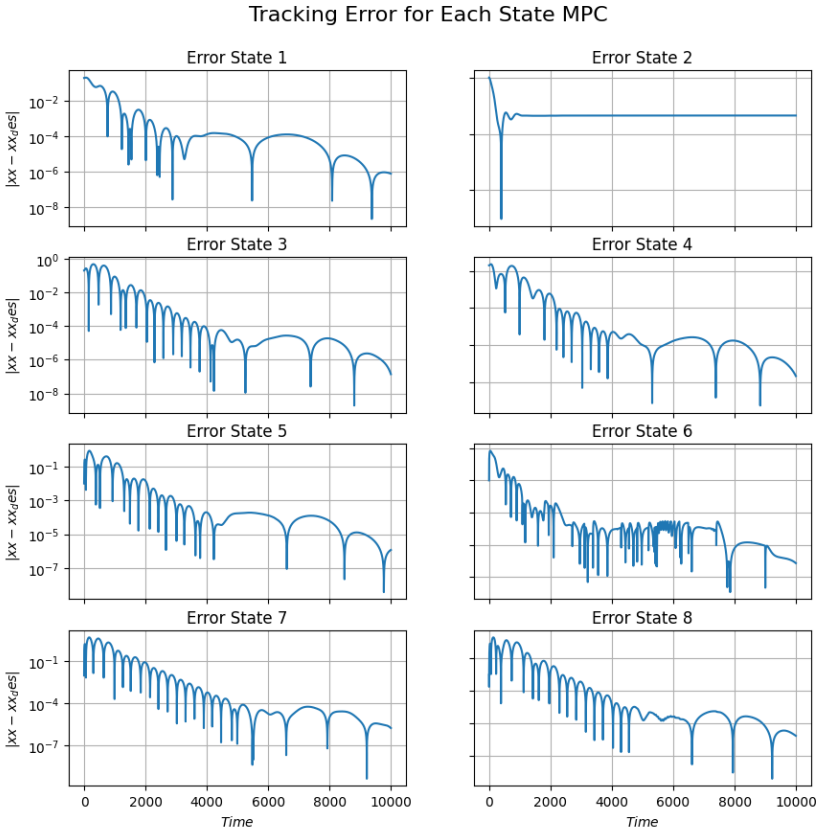
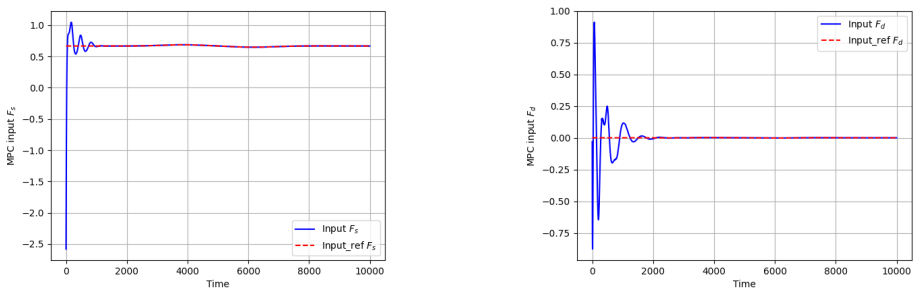


Figure 5.4: Tracking error over time with MPC controller ”Logarithmic scale"



(a) Input F_s (b) Input F_d

Figure 5.5: MPC Control Input vs Optimal Input

5.3.1 Significance of Results

The results illustrate that MPC adapts effectively to disturbances by modifying the control inputs in real time. Figures 5.5 demonstrate how the inputs adjust dynamically to compensate for perturbations, eventually converging back to the nominal optimal inputs.

5.4 Summary

MPC has proven to be a robust strategy for quadrotor trajectory tracking, particularly under the influence of disturbances. Its ability to handle multivariable dynamics and enforce constraints in real time makes it a powerful tool for aerial robotics.

5.5 Conclusion

This chapter demonstrated the application of Model Predictive Control to a quadrotor system. By continuously solving finite-horizon optimization problems, MPC achieves accurate trajectory tracking and effective disturbance rejection. Compared to classical approaches, MPC provides the additional advantage of constraint handling, which is essential for practical deployment in real-world quadrotor missions.

5.6 Future Research Directions

The analysis presented in this work highlights several promising directions for further research in the field of quadrotor optimal control. These extensions would enhance the robustness, efficiency, and applicability of the proposed methodologies:

1. **Integration of Learning-Based Methods:** The incorporation of machine learning techniques, such as reinforcement learning, into the MPC and LQR frameworks could improve adaptability to uncertain or unstructured environments. Hybrid schemes that combine data-driven models with physics-based dynamics may increase robustness to modeling inaccuracies.

2. **Energy Efficiency Optimization:** While the present work focuses on trajectory tracking and stability, future research may address the optimization of energy consumption. Minimizing energy usage is particularly relevant for battery-powered aerial vehicles and could be achieved by explicitly including energy-related costs in the control formulation.

3. **Robustness under Environmental Variability:** A natural extension of this study is to assess controller performance under diverse environmental conditions, such as wind disturbances or varying payloads. Robust MPC formulations or stochastic optimization approaches may be required to guarantee reliable operation.

4. **Scalability to Multi-Agent Systems:** Extending the proposed control strategies to multi-quadrotor systems would enable cooperative behaviors such as formation flying and payload transport. The design of distributed MPC or decentralized LQR schemes represents an important challenge in scalability and coordination.

5. **Experimental Validation:** Real-world testing remains a critical step for bridging the gap between theoretical analysis and practical deployment. Hardware-in-the-loop simulations and experimental prototypes would allow assessment of computational feasibility, sensor noise resilience, and actuator limitations.

In summary, these directions emphasize the importance of advancing control strategies beyond simulation-based studies. By integrating learning-based methods, energy-aware formulations, robustness analysis, scalability, and real-world validation, future work can significantly enhance the reliability and applicability of optimal control in aerial robotics.

Conclusions

This report has presented a comprehensive study optimal control techniques applied to a quadrotor with a suspended load. The work began with the formulation of the system dynamics and applied Newton's method for trajectory generation. The method proved effective in computing optimal transitions between equilibrium points, with smoother reference curves leading to faster convergence and reduced cost.

The Linear Quadratic Regulator (LQR) was then implemented for trajectory tracking. Simulation results confirmed its ability to stabilize the system and compensate for disturbances with minimal computational effort, demonstrating its suitability for real-time applications.

Finally, Model Predictive Control (MPC) was investigated as a predictive and constraint-aware strategy. MPC showcased strong performance in handling disturbances and respecting system constraints, offering flexibility and robustness, though at the cost of higher computational requirements.

Overall, the combination of these methods highlights the trade-offs between efficiency, robustness, and computational demand. Newton's method is well-suited for open-loop trajectory design, LQR provides reliable feedback control, and MPC offers predictive capabilities for complex and constrained scenarios. Together, they establish a versatile framework for addressing the challenges of quadrotor control and provide a foundation for future research and practical deployment in aerial robotics.

Bibliography

[1] Dimitri P. Bertsekas. *Nonlinear Programming.* Athena Scientific, 2016.

[2] Arthur E. Bryson and Yu-Chi Ho. *Applied Optimal Control: Optimization, Estimation, and Control.* Taylor & Francis, 1975.

[3] G. Notarstefano. Lecture slides on optimal control. Lecture notes distributed in Optimal Control course at Università di Bologna, 2023. Accessed on September 19, 2023.

[4] Mayuresh Pachore. Optimal control exam project group 11. https://github.com/pachoremayuresh/Optimal_Control_Exam_Project_Group_11, 2023. Online; accessed 2 September 2025.